SpecForge — UniHack Master Roadmap

AI-Powered Product Intelligence for Industrial Commerce

Team FasterThanAI · Priyanshu · Nupur · Raj

Contents

PART 0 · THE DECISION

What you are building

SpecForge — an AI product-intelligence engine that turns the three fields a manufacturer actually gives a distributor into a complete, validated, commerce-ready catalog record, with every single value traceable back to the exact document page it came from.

> *Rename it if you like. Keep the tagline:* ***"Every attribute, with a receipt."***

The verdict on the lead agent

You are **not** submitting the lead agent, and you are **not** starting from zero. You are keeping the machine and replacing the domain.

	Count	Decision
Services **deleted**	9	apollo, email_guesser, followup, gmail, hunter, reply_classification, reply_response, vapi, opportunity*
Routes **deleted**	10	calls, emails, followups, gmail, hunter, apollo, replies, reply_classification, reply_responses, lead_agent
Tables **dropped**	7	EmailDraft, FollowUpDraft, ReplyResponseDraft, CallLog, CallScript, GmailToken, GmailOAuthState
Frontend pages **deleted**	2	Calls.jsx, Emails.jsx
Services **reused untouched**	6	scraper, document, embedding, knowledge, database, time_utils
Services **renamed + re-prompted**	3	lead_research → enrichment, lead_scoring → quality, lead_discovery → sourcing
UI primitives **reused untouched**	9 + theme	Button, Card, Badge, Table, PageHeader, EmptyState, Skeleton, Toast, StatCard

* `opportunity_service.py` is optionally repurposed as the AI schema generator — see Phase 9.

Net effect: roughly 40% of the codebase is deleted, roughly 60% of what remains is reused as-is or renamed, and your entire week goes into the four things that actually score.

PART 1 · PROBLEM STATEMENT

Unilog's world

Unilog is an AI-native B2B commerce and PIM company serving roughly 400 distributors and manufacturers across North America — plumbing, HVAC, electrical, industrial supply. Their content library covers **11M+ actively managed SKUs**.

Their own marketing names two gaps:

- **The Trust Gap** — when product data is incomplete or inaccurate, buyers cannot find what they need online, so they pick up the phone.
- **The Efficiency Gap** — teams spend their time on manual spec lookups instead of selling.

The challenge, restated

> Industrial manufacturers manage vast product information across websites, catalogs, technical documents and digital assets. Transforming this fragmented data into accurate, structured, commerce-ready product intelligence is complex and time-consuming.
>
> Build an AI-powered solution that automates the **creation, enrichment and validation** of product intelligence from **limited product information**.

Expected outcomes:

1. Generate structured product intelligence from limited inputs
2. Improve product data quality and consistency
3. Validate and enrich information with traceable outputs
4. Scale efficiently across large product catalogs

Our framing — the sentence that opens the deck

> A distributor onboarding a new manufacturer receives 5,000 SKUs as a spreadsheet with a part number, a brand and a one-line description — plus a folder of PDF spec sheets nobody has time to read. Turning that into sellable catalog data costs 15–20 minutes of skilled human attention per SKU. That is **1,500 hours** for one manufacturer. SpecForge does it in minutes and shows its work.

This framing matters because it is **the real workflow**. Content arrives at a distributor as documents from manufacturers — it is not scraped off the open web. Building document-first is both more defensible and technically safer.

PART 2 · THE SOLUTION

The transformation, concretely

Input — one row of a manufacturer's onboarding spreadsheet:

```
part_number:        70-104-01
brand:              Apollo
short_description:  Ball valve, bronze
```

Plus, optionally, whatever documents came with it — a spec-sheet PDF, a scanned catalog page, a product URL.

Output — a complete catalog record:

```
ATTRIBUTE           RAW             NORMALISED       CONF   SOURCE
body_material       Bronze          bronze            96    spec.pdf p.2
size_nominal        1/2 in          12.7 mm           94    spec.pdf p.2
pressure_rating     600 WOG         600 psi           91    product page
temp_range_min      -20°F           -28.9 °C          88    spec.pdf p.3
end_connection      NPT Female      npt_female        93    product page
port_type           Full Port       full_port         85    catalog.jpg (vision)
handle_material     SS / Steel      —                 41    ⚠ CONFLICT (2 sources)
agency_approvals    UL, CSA         ul; csa           90    spec.pdf p.4
… 12 more

completeness  84%    ·    mean confidence  87    ·    grade  B    ·    3 need review
```

Every value carries **where it came from, how it was extracted, how confident the system is, and whether a rule flagged it**. Low-confidence and conflicting values go to a human review queue; the rest are bulk-approved.

The five agents

The brief names *AI agents* first. Architect and present it that way — it is also genuinely the right decomposition.

Agent	Job	Built from
Sourcing Agent	Gather every available source for a part — uploaded documents, product URLs, images	`sourcing_service` (was lead_discovery)
Extraction Agent	Pull candidate attribute	`enrichment_service` (was

Agent	Job	Built from
	values from text and images against the target schema	lead_research)
Normalisation Agent	Convert raw values to canonical units, enums and formats. **Deterministic — no LLM**	new
Validation Agent	Rule checks from schema + AI plausibility pass + cross-source conflict detection	new
Review Agent	Score confidence and completeness, route uncertain values to a human, learn from decisions	`quality_service` (was lead_scoring) + approval state machine

An **Orchestrator** runs them per product as a background job, writing progress to a jobs table.

PART 3 · APPROACH & ARCHITECTURE

The pipeline

```
1  INGEST       CSV / XLSX of minimal fields  +  document uploads
                     |
                     v
2  SOURCE       resolve every available source per part
                 · uploaded PDF / DOCX / image
                 · manufacturer product URL (secondary)
                     |
                     v
3  EXTRACT      HTML  → visible text  → Gemini → candidate attributes
                PDF   → pypdf text    → Gemini → candidate attributes
                scan  → page image    → Gemini VISION → candidate attributes
                     |
                     v
4  NORMALISE    units · fractions · enums · booleans · thread specs
                DETERMINISTIC PYTHON. No model in this step.
                     |
                     v
5  VALIDATE     rule checks from AttributeSchema
                + AI plausibility pass
                + cross-source conflict detection
                     |
                     v
6  SCORE        per-attribute confidence · per-product completeness · grade
                     |
                     v
7  REVIEW       human approves / edits / rejects · bulk actions
                     |
```

```
                    v
8  EXPORT       clean structured catalog → CSV / JSON, with provenance column
```

Stages 1, 2, 3 (text), 6 and 7 exist in your codebase today in another domain. Stages 4, 5, 8 and vision are new — and they are exactly the four things that score.

Architecture diagram

Redraw this in draw.io for slide 9. The structure is what matters.

```
+-------------------------------------------------------------------+
|  REACT 19 + VITE + TAILWIND  (Vercel)         Midnight Aurora UI  |
|  Catalogs . Products . Sources . Review Queue . Schema . Dashboard|
+-------------------------------+-----------------------------------+
                                |  axios / REST / JSON
+-------------------------------v-----------------------------------+
|  FASTAPI + UVICORN  (Render)                                      |
|  +-------------------------------------------------------------+  |
|  |  API LAYER    routes/ . Pydantic validation . CORS . DI     |  |
|  +-----------------------------+-------------------------------+  |
|  +-----------------------------v-------------------------------+  |
|  |  ORCHESTRATOR   BackgroundTasks . EnrichmentJob progress table |  |
|  +-----------------------------+-------------------------------+  |
|  +---------+---------+--------v--------+-----------+-------------+ | |
|  | Sourcing| Extract | Normalisation   | Validation|   Review    | | |
|  |  Agent  |  Agent  |  Agent (rules)  |   Agent   |   Agent     | | |
|  +----+----+----+----+-----------------+-----+-----+------+------+ | |
+-------|---------|-----------------------|-----------|-------------+ |
        |         |                       |           |
   +----v----+ +--v----------+     +------v-----+ +---v---------+
   | Docs    | | GEMINI      |     | RULE       | | POSTGRES    |
   | pypdf   | | 2.5 Flash   |     | ENGINE     | | + pgvector  |
   | docx    | | text+VISION |     | schema     | | 17 tables   |
   | scraper | | embed-001   |     | driven     | | provenance  |
   +---------+ +------+------+     +------------+ +-------------+
                      |
              +-------v------------------------+
              |  RAG   taxonomy . reference    |
              |  specs . brand rules           |
              |  hybrid semantic + keyword     |
              +--------------------------------+
```

Data model

Three new tables. Everything else is a rename of something you already have.

```
Catalog                     -- was Campaign
  id · name · vertical · description · created_at

AttributeSchema             -- NEW
  id · catalog_id · category_name              -- "Ball Valve"
  attributes JSON [{ key, label, data_type, unit_family,
                     allowed_values, required, min, max }]

Product                     -- was Lead
  id · catalog_id
  part_number · manufacturer · short_description     -- THE MINIMAL INPUT
  category · canonical_name · long_description
  status          pending|sourcing|enriching|needs_review|approved|failed
  completeness_score · confidence_score · quality_grade
```

```
  enriched_at · model_used · error

ProductAttribute            -- NEW · the heart of the system
  id · product_id
  key · value_raw · value_norm · unit
  confidence                                    -- 0-100
  status          proposed|approved|rejected|conflicted
  source_id       → SourceDocument              -- PROVENANCE
  extraction_method   html|pdf|vision|inferred
  validation_flags JSON
  model_used · reviewed_by · reviewed_at

SourceDocument              -- was DiscoveredLead
  id · product_id · url · filename
  doc_type        html|pdf|image|xlsx
  fetched_at · content_hash · text_snippet · page_number

AttributeConflict           -- NEW · multi-source verification
  id · product_id · key
  candidates JSON [{ value, source_id, confidence }]
  resolution      unresolved|auto|human
  resolved_value · resolved_by · resolved_at

EnrichmentJob               -- copy LeadResearchJob verbatim
  id · catalog_id · status · total · processed · succeeded · failed
  started_at · finished_at · error
```

Why attributes are rows, not a JSON blob: the rubric demands per-attribute confidence, per-attribute provenance and per-attribute review. A blob throws all three away. This single decision is what makes the trust story provable rather than claimed.

Technology stack — slide 10

Layer	Technology	Why
Frontend	React 19 · Vite 8 · Tailwind 4 · React Router 7 · Recharts · Framer Motion	Component reuse across 6 pages; instant theming
Backend	Python 3.12 · FastAPI · Uvicorn · Pydantic 2	Async-native for concurrent model calls; validation at the boundary; auto Swagger
Database	PostgreSQL (Supabase) · SQLAlchemy 2 · **pgvector**	Relational integrity + native vector search in one store
AI — generation	**Gemini 2.5 Flash** (text + vision)	Multimodal in one SDK; cost-efficient at 4+ calls per SKU
AI — retrieval	**gemini-embedding-001**, 3072-dim	Semantic taxonomy matching and reference lookup
Documents	pypdf · python-docx · BeautifulSoup4	PDF, Word and HTML ingestion
Async	FastAPI BackgroundTasks + job tables	Catalog-scale processing without HTTP timeouts

Layer	Technology	Why
Deploy	Vercel (frontend) · Render (backend) · Supabase (DB)	Zero-ops, free tier

PART 4 · EVALUATION — SLIDE 4 ANSWERED

Template slide 4 is the scoring rubric wearing a disguise. These are the answers, written to be pasted almost verbatim. **Q2 is the longest question and enumerates five validation approaches — that is where the marks concentrate. Answer all five explicitly.**

Q1 · How does your solution enrich minimal product information?

SpecForge takes the three fields a manufacturer actually supplies — **part number, brand, short description** — and treats them as a search key, not as data.

The **Sourcing Agent** resolves every available source for that part: uploaded spec-sheet PDFs, scanned catalog pages, product URLs. The **Extraction Agent** reads each source against a category-specific attribute schema, using text extraction for digital documents and **Gemini vision** for scans and images. Each candidate value is written as its own record carrying its source, its extraction method and a confidence score.

A single row with 3 fields becomes **20+ structured, typed, unit-normalised attributes** plus a generated long description and search synonyms — typically in under 90 seconds per SKU, running hundreds in parallel as a background job.

Q2 · How does your solution ensure accuracy and trust?

Five independent layers, deliberately mixing deterministic and probabilistic checks.

1 · Confidence scoring. Every attribute carries a 0–100 confidence derived from extraction agreement, source authority (manufacturer spec sheet outranks a marketing page) and rule-check outcomes. Product-level scores aggregate to a completeness percentage and an A–D quality grade.

2 · Multi-source verification. Each part is extracted from every available source independently. Where sources agree, confidence rises. Where they disagree, we **do not silently pick a winner** — an `AttributeConflict` record is created holding every candidate with its source, and it is escalated.

3 · Rule-based validation. The category `AttributeSchema` defines data type, unit family, allowed values and numeric ranges. Every extracted value is checked deterministically: type match, enum membership, range bounds, unit-family correctness. Failures are recorded in `validation_flags` and reduce confidence. **No model is involved in this step — arithmetic and unit conversion are done in Python, never by an LLM.**

4 · AI validation. A separate Gemini pass reviews the assembled attribute set for domain plausibility — a bronze valve rated to 3000 psi is type-valid but physically implausible. This catches what rules cannot express.

5 · Human-in-the-loop review. Nothing is published automatically. Attributes enter as `proposed`; a reviewer bulk-approves high-confidence values and hand-adjudicates flagged and conflicting ones. Every decision records who and when.

Underneath all five: full provenance. Every attribute links to its `SourceDocument` — file, page number and text snippet. In the UI you click any value and land on the exact PDF page it came from. **Trust is demonstrated, not asserted.**

Q3 · What makes your solution scalable for enterprise product catalogs?

Large catalogs. Enrichment runs as asynchronous background jobs, not HTTP requests. A job returns immediately with an ID; workers process the catalog and write progress to a jobs table the UI polls. The same architecture handles 25 SKUs or 25,000 — only wall-clock changes. Batch size, concurrency and per-source timeouts are configuration.

New manufacturers. Onboarding a manufacturer requires **no code**. Upload their documents or add a source URL pattern; the Sourcing Agent handles the rest. Nothing about extraction is manufacturer-specific.

Different document formats. A single ingestion interface behind format-specific readers — pypdf for digital PDFs, python-docx for Word, BeautifulSoup for HTML, Gemini vision for scans and images. Adding a format means adding one reader, not touching the pipeline.

Continuous updates. Every source stores a `content_hash`. Re-running enrichment re-fetches, compares hashes and only re-extracts what changed — then diffs new values against approved ones and raises changes for review rather than overwriting. Spec revisions become a review queue item, not a silent data corruption.

Cost. Roughly four Gemini Flash calls per SKU, batched and cached by content hash. A 5,000-SKU catalog costs single-digit dollars in inference.

PART 5 · THE DECK — ALL 15 SLIDES

Fill the official template. This is the content for each slide.

Slide 1 · Guidelines

Leave as provided.

Slide 2 · Team Details

```
Team name:          FasterThanAI
Team leader name:   Priyanshu Kumar
Members:            Nupur · Raj
```

Slide 3 · Brief about your solution

SpecForge — every attribute, with a receipt.

An AI product-intelligence engine that turns the minimum a manufacturer supplies — part number, brand, one-line description — into a complete, validated, commerce-ready catalog record.

Five specialised agents source the available documents, extract attributes from text and scanned images, normalise units deterministically, validate against schema rules and cross-check between sources, then route anything uncertain to a human reviewer.

Every published value is traceable to the exact document page it came from. A distributor onboarding 5,000 SKUs currently spends ~1,500 hours of skilled human time. SpecForge reduces that to a review queue.

Slide 4 · The three questions

Paste **Part 4** above. This is the highest-value slide in the deck — give it the most space.

Slide 5 · Opportunities

How is it different from existing ideas?

Most enrichment tools are a prompt wrapped in an interface: text in, JSON out, no way to know whether it is right. SpecForge treats **provenance and validation as the product**, not as a feature. Three specific differences:

- **Attribute-level records, not document-level blobs.** Every value is independently sourced, scored, validated and reviewable.
- **Deterministic normalisation.** Units, fractions and enums are converted in Python. We never ask a language model to do arithmetic — the single most common failure mode in this category.
- **Disagreement is surfaced, not resolved silently.** Conflicting sources produce a visible conflict record, not a coin flip hidden behind a confidence number.

How does it solve the problem statement?

It addresses all four expected outcomes directly: it generates structured intelligence from three input fields; it improves consistency through deterministic normalisation and schema validation; it makes every output traceable to a source page; and it scales through asynchronous job processing that is format- and manufacturer-agnostic.

USP

The only enrichment pipeline where you can click any attribute and land on the exact page of the exact document it came from. Auditable AI, not trust-me AI.

Slide 6 · Features

- Minimal-input ingestion — CSV / XLSX of part number, brand, description
- Multi-format document ingestion — PDF, DOCX, scanned images, HTML
- **Vision extraction** from scanned spec sheets and catalog pages
- Category attribute schemas with types, units, enums and ranges
- Five-agent enrichment pipeline with an async orchestrator
- **Deterministic unit and format normalisation** — imperial/metric, fractions, threads, materials
- Rule-based validation with per-attribute flags
- AI plausibility validation
- **Multi-source conflict detection and adjudication**
- Per-attribute confidence and provenance to page level
- Product completeness scoring and A–D quality grading
- Human review queue with bulk approve, inline edit and reject
- RAG over taxonomy and reference specifications
- Change detection via content hashing for continuous updates
- Clean catalog export — CSV / JSON with provenance
- Cross-reference matching to find equivalent parts *(stretch)*
- Search synonym generation for findability *(stretch)*

Slide 7 · Process flow

Redraw the 8-stage pipeline from Part 3 as a horizontal flow. Show the branch at stage 3 (text path vs vision path) and the loop back from stage 7 to stage 5 on rejection.

Slide 8 · Wireframes *(optional)*

Include if you have time — the Review Queue is the screen worth showing.

Slide 9 · Architecture

Redraw the diagram from Part 3 in draw.io. Keep the five agents visible as distinct boxes; that is what earns the “AI agents” line in the brief.

Slide 10 · Technologies

Paste the stack table from Part 3.

Slide 11 · Estimated cost *(optional — include it, most teams will not)*

Item	Monthly at MVP	At 50k SKU/month
Gemini 2.5 Flash — ~4 calls/SKU	Free tier	~$40–70
Gemini embeddings	Free tier	~$5
Supabase Postgres + pgvector	Free tier	$25
Render backend	Free tier	$7–25
Vercel frontend	Free tier	$0–20
Total	**$0**	**~$80–145/month**

> Against ~1,500 hours of manual work per 5,000-SKU manufacturer onboarding, payback is immediate.

Slide 12 · MVP snapshots

Five screenshots: the Products table mid-enrichment, a product detail with attributes and confidence, an attribute expanded to its source PDF page, the conflict adjudication view, the Review Queue.

Slide 13 · Future development

- Learning loop — reviewer decisions become few-shot examples that raise confidence over time
- ERP and PIM connectors — direct write-back to Unilog CX1, SAP, Epicor
- Standard taxonomy mapping — ETIM, UNSPSC, eCl@ss
- Cross-reference engine — equivalent parts across manufacturers
- Digital-asset intelligence — auto-tag and validate product imagery
- Bulk change-monitoring — scheduled re-crawl with diff review
- Multi-tenant with role-based access and audit logging

Slide 14 · Links

```
GitHub Public Repository:   https://github.com/<org>/specforge
Demo Video (3 minutes):     <link>
Working Prototype:          https://specforge.vercel.app
```

Before submitting, verify all three open in a private browsing window. A private repo or a sleeping backend scores zero on that slide.

PART 6 · BUILD ROADMAP

Eleven phases. Each has a **gate** — do not start the next phase until the gate passes. Time is not the constraint, so the constraint becomes discipline: a half-finished phase 5 is worth less than a finished phase 4.

Phase	Deliverable	Gate
0	Repo created, dead code stripped	App boots, `/docs` lists only surviving routes
1	New data model + migrations	All 8 tables exist in Postgres and SQLite
2	Ingestion — CSV + document upload	25 products and their PDFs are in the DB
3	Extraction Agent (text)	One product yields 15+ attributes with sources
4	Normalisation Agent	`1/2"` and `12.7mm` produce the same `value_norm`
5	Validation Agent + conflicts	A deliberate bad value is flagged; a disagreement raises a conflict
6	Vision extraction	A scanned page with no text layer yields attributes
7	Quality scoring	Every product shows completeness %, mean confidence, grade
8	Review Queue UI	A human can bulk-approve, edit and reject
9	Export + Dashboard + schema generator	CSV out with provenance; dashboard shows funnel
10	Deploy + seed + harden	Live URL, warm, 25 fully enriched demo products

Team split. Priyanshu owns phases 0, 1, 2, 5, 9, 10 (backend spine). Raj owns 3, 4, 6, 7 (the AI layer). Nupur owns 8, plus the deck, demo data curation and the video. Phases 3–7 can run in parallel with 8 once phase 2 lands.

Before phase 0 — two non-negotiables:

1. **Check the eligibility rules.** Find out whether UniHack requires code written during the event. Use a fresh repo either way, and state plainly in the README that SpecForge reuses an internal framework your team wrote previously. Disclosure costs nothing; discovery costs everything.

2. **Nupur curates real data.** 25–30 genuine part numbers across exactly 3 categories, with their real spec-sheet PDFs downloaded. Suggested categories: **ball valves, pressure gauges, pipe fittings**. Everything downstream is designed against this data, so get it first.

PART 7 · AI AGENT PROMPTS

Paste these into **Codex** or **Antigravity** in order, one phase at a time. Each is self-contained.

Rules for using them. Run one phase per session. Read the diff before accepting. If the agent proposes touching a file the prompt marked as frozen, reject and re-prompt. Commit at every gate.

PROMPT 0 · Repo setup and strip

```
You are working on a Python + React monorepo that is being repurposed.

CONTEXT
The repo currently implements "AI Lead Generation MVP" — a B2B sales outreach tool.
Backend: FastAPI + SQLAlchemy 2 + Pydantic 2, in backend/app/ with the layout
  api/routes/*.py   thin HTTP handlers
  services/*.py     all business logic
  db/models.py      SQLAlchemy models
  db/database_utils.py  ensure_*_columns() startup migrations
  schemas/*.py      Pydantic request/response models
Frontend: React 19 + Vite 8 + Tailwind 4 in frontend/src/.

We are repurposing it into "SpecForge", a product-data enrichment engine for
industrial distributors. Sales outreach is being removed entirely.

TASK — delete dead code only. Add nothing.

1. Delete these service files:
   apollo_service.py, email_guesser_service.py, followup_service.py,
   gmail_service.py, hunter_service.py, reply_classification_service.py,
   reply_response_service.py, vapi_service.py

2. Delete these route files:
   calls.py, emails.py, followups.py, gmail.py, hunter.py, apollo.py,
   replies.py, reply_classification.py, reply_responses.py, lead_agent.py
   Remove their imports and include_router() calls from api/api_router.py.

3. Delete these Pydantic schema files and any imports of them:
   email_schema.py, followup_schema.py, reply_response_schema.py, call_schema.py

4. In db/models.py delete these model classes and every relationship that
   references them:
   EmailDraft, FollowUpDraft, ReplyResponseDraft, CallLog, CallScript,
   GmailToken, GmailOAuthState
   Delete the matching ensure_*_columns functions in db/database_utils.py and
   their calls in main.py.

5. Delete frontend/src/pages/Calls.jsx and frontend/src/pages/Emails.jsx.
   Remove their routes from App.jsx and their nav entries from
```

```
   components/Sidebar.jsx.

6. Remove now-unused dependencies from backend/requirements.txt:
   google-api-python-client, google-auth-oauthlib, google-auth-httplib2,
   dnspython, tenacity
   (tenacity is currently imported nowhere — verify with grep before removing.)

DO NOT TOUCH
  services/scraper_service.py, services/document_service.py,
  services/embedding_service.py, services/knowledge_service.py,
  db/database.py, utils/time_utils.py,
  frontend/src/components/ui/*, frontend/src/index.css, tailwind.config.js

ACCEPTANCE
  - `uvicorn app.main:app` starts with no import errors
  - GET /docs lists only: health, dashboard, campaigns, leads, lead_scoring,
    discovery, knowledge, ai, opportunities, analytics
  - `npm run build` in frontend/ succeeds
  - grep -ri "gmail\|vapi\|hunter\|apollo\|follow_up\|reply_" backend/app
    returns nothing outside comments

Report every file you deleted and every import you removed.
```

PROMPT 1 · The new data model

```
CONTEXT
SpecForge repo (FastAPI + SQLAlchemy 2 + Pydantic 2). Backend at backend/app/.
Schema changes are applied by ensure_*_columns(engine) functions in
db/database_utils.py, called from main.py at startup. There is no Alembic.
Study ensure_lead_research_job_columns() in db/database_utils.py — it inspects
the engine, branches on dialect (postgresql vs sqlite) for TIMESTAMP/DATETIME
and SERIAL/AUTOINCREMENT, CREATEs the table if missing, then ALTERs in any
missing columns. Follow that pattern exactly for every new table.

TASK — rename the domain and add three new tables.

RENAME (models + all references, keep every column unless stated)
  Campaign  -> Catalog        table campaigns -> catalogs
      keep id, name (was campaign_name), created_at
      add  vertical VARCHAR(255), description TEXT
      drop industry, location, target_role, offer
  Lead      -> Product        table leads -> products
      keep id, created_at
      rename campaign_id -> catalog_id
      add  part_number VARCHAR(255) NOT NULL INDEX
           manufacturer VARCHAR(255)
           short_description TEXT
           category VARCHAR(255)
           canonical_name VARCHAR(500)
           long_description TEXT
           status VARCHAR(50) DEFAULT 'pending'
           completeness_score INTEGER
           confidence_score INTEGER
           quality_grade VARCHAR(2)
           enriched_at TIMESTAMP
           model_used VARCHAR(255)
           error TEXT
      drop every sales column: company_name, website, industry, location,
           contact_name, contact_role, email, phone, source, call_status,
           last_call_outcome, last_called_at, do_not_call, ai_* , research_*
  LeadResearchJob -> EnrichmentJob   table lead_research_jobs -> enrichment_jobs
      rename campaign_id -> catalog_id, total_leads -> total, researched -> succeeded
```

```
  DiscoveredLead  -> SourceDocument  table discovered_leads -> source_documents
      keep id, created_at, updated_at
      replace all contact columns with
           product_id INTEGER INDEX NOT NULL
           url VARCHAR(1000)
           filename VARCHAR(500)
           doc_type VARCHAR(20)          -- html|pdf|image|xlsx|docx
           fetched_at TIMESTAMP
           content_hash VARCHAR(64) INDEX
           text_snippet TEXT
           page_number INTEGER

DELETE these models entirely: DiscoveryJob, LeadScoringJob,
EmailExtractionJob, Opportunity

KEEP UNCHANGED: KnowledgeDocument, CompanyKnowledge

CREATE three new models

  AttributeSchema        table attribute_schemas
    id INTEGER PK
    catalog_id INTEGER FK catalogs.id INDEX
    category_name VARCHAR(255) NOT NULL
    attributes TEXT NOT NULL      -- JSON array, see shape below
    created_at, updated_at
    -- attributes JSON shape:
    -- [{ "key":"pressure_rating", "label":"Pressure Rating",
    --    "data_type":"number|string|enum|boolean",
    --    "unit_family":"pressure|length|temperature|mass|none",
    --    "allowed_values":[], "required":true, "min":null, "max":null }]

  ProductAttribute       table product_attributes
    id INTEGER PK
    product_id INTEGER FK products.id INDEX NOT NULL
    key VARCHAR(255) NOT NULL INDEX
    value_raw TEXT
    value_norm TEXT
    unit VARCHAR(50)
    confidence INTEGER            -- 0-100
    status VARCHAR(20) DEFAULT 'proposed'  -- proposed|approved|rejected|conflicted
    source_id INTEGER FK source_documents.id
    extraction_method VARCHAR(20)          -- html|pdf|vision|inferred
    validation_flags TEXT                  -- JSON array of strings
    model_used VARCHAR(255)
    reviewed_by VARCHAR(255)
    reviewed_at TIMESTAMP
    created_at, updated_at
    UNIQUE (product_id, key, source_id)

  AttributeConflict      table attribute_conflicts
    id INTEGER PK
    product_id INTEGER FK products.id INDEX NOT NULL
    key VARCHAR(255) NOT NULL
    candidates TEXT NOT NULL      -- JSON [{value, source_id, confidence}]
    resolution VARCHAR(20) DEFAULT 'unresolved'  -- unresolved|auto|human
    resolved_value TEXT
    resolved_by VARCHAR(255)
    resolved_at TIMESTAMP
    created_at

Set up SQLAlchemy relationships: Catalog 1-N Product, Product 1-N
ProductAttribute, Product 1-N SourceDocument, Product 1-N AttributeConflict,
SourceDocument 1-N ProductAttribute. Use cascade="all, delete-orphan" on the
Catalog->Product and Product->* sides.
```

```
Write ensure_*_columns() for all eight tables and call them from main.py in
dependency order (catalogs, attribute_schemas, products, source_documents,
product_attributes, attribute_conflicts, enrichment_jobs, then the two
knowledge tables).

Rename the route modules and update api_router.py:
  routes/campaigns.py -> routes/catalogs.py   prefix /catalogs
  routes/leads.py     -> routes/products.py   prefix /products
  routes/discovery.py -> routes/sources.py    prefix /sources
  routes/lead_scoring.py -> routes/quality.py prefix /quality
Delete routes/ai.py and routes/opportunities.py for now.
Strip any handler that references a deleted column; leave a stub returning 501
rather than deleting a whole route file.

DO NOT TOUCH
  services/*.py in this phase. Data model only.

ACCEPTANCE
  - App boots against SQLite with no tables present and creates all eight
  - App boots against an existing Postgres and adds only missing columns
  - GET /docs shows /catalogs, /products, /sources, /quality, /knowledge
  - python -c "from app.db.models import *" imports cleanly
```

PROMPT 2 · Ingestion

```
CONTEXT
SpecForge. Models from phase 1 exist. services/document_service.py already
provides extract_text_from_file(file_path, file_type) and
chunk_text(text, max_chars, overlap) — reuse them, do not rewrite.
python-multipart is already installed for uploads.

TASK — build ingestion.

1. services/ingestion_service.py

   parse_product_csv(file_bytes) -> list[dict]
     Accept CSV or XLSX. Tolerate messy real-world headers by fuzzy-matching
     column names case-insensitively, ignoring spaces/underscores/hyphens:
       part_number  <- part, partno, part no, sku, item, item number, mfr part
       manufacturer <- brand, mfr, mfg, make, vendor, supplier
       short_description <- description, desc, short desc, name, title
     Return normalised dicts. Rows missing part_number are returned in a
     separate "rejected" list with a reason. Never raise on a bad row.

   ingest_products(db, catalog_id, rows) -> dict
     Upsert on (catalog_id, part_number). Existing products are updated, not
     duplicated. Return {created, updated, rejected, total}.

   register_document(db, product_id, filename, file_bytes, doc_type) -> SourceDocument
     Compute sha256 into content_hash. If a SourceDocument with the same
     content_hash already exists for that product, return it instead of
     creating a duplicate. Extract text via document_service and store the
     first 2000 chars in text_snippet. Save the file under
     storage/sources/{product_id}/{content_hash}{ext} — create dirs as needed.

2. routes/products.py
   POST /products/import          multipart CSV/XLSX + catalog_id -> ingest report
   GET  /products                 list, filters: catalog_id, status, q,
                                  min_confidence, needs_review(bool)
                                  paginated: limit(default 50, max 200), offset
   GET  /products/{id}            product + attributes + sources + conflicts
```

```
   POST /products/{id}/documents  multipart upload, 1..N files
   DELETE /products/{id}

3. routes/catalogs.py
   POST /catalogs, GET /catalogs, GET /catalogs/{id}, DELETE /catalogs/{id}

Pydantic schemas in schemas/product_schema.py and schemas/catalog_schema.py.

FILE SAFETY
  Accept only .csv .xlsx .pdf .docx .txt .md .png .jpg .jpeg .webp
  Reject anything above 25 MB with a 413.
  Sanitise filenames — strip path separators, never trust the client name.

DO NOT TOUCH  services/document_service.py, services/scraper_service.py

ACCEPTANCE
  - POST a 25-row CSV with messy headers -> 25 products created
  - POST the same CSV again -> 0 created, 25 updated
  - Upload the same PDF twice -> one SourceDocument, not two
  - GET /products?needs_review=true returns an empty list (nothing enriched yet)
```

PROMPT 3 · Extraction Agent (text)

```
CONTEXT
SpecForge. Gemini is accessed through the google-genai SDK; see
services/ai_service.py for the client setup and for two helpers you must
reuse: extract_json_from_text(text) and clean_value(value).
services/knowledge_service.py provides search_relevant_knowledge(db, query, limit)
for RAG. Settings live in core/config.py (GEMINI_API_KEY, GEMINI_MODEL).

TASK — create services/extraction_service.py, the Extraction Agent.

  load_schema(db, catalog_id, category) -> dict
    Return the AttributeSchema.attributes JSON for the category, or a generic
    fallback schema if none is defined.

  build_extraction_prompt(product, schema, source_text, source_label) -> str
    The prompt must:
      - state the product identity (part number, manufacturer, description)
      - list every schema attribute with its key, data type, unit family and
        allowed values
      - include the source text, truncated to 12000 characters
      - include up to 3 relevant knowledge chunks retrieved via
        search_relevant_knowledge for taxonomy guidance
      - demand a strict JSON array and nothing else:
        [{"key":"...", "value_raw":"...", "confidence":0-100,
          "evidence":"the exact phrase from the source that supports this"}]
      - instruct: extract ONLY what is explicitly present. Never infer,
        never guess, never fill from general knowledge about the brand.
        Omit an attribute entirely rather than inventing it.
        Copy value_raw verbatim from the source, including its unit.

  extract_from_source(db, product, source_document) -> list[dict]
    Call Gemini, parse with extract_json_from_text, validate each item has a
    key present in the schema, clamp confidence to 0-100, drop malformed rows.
    Return candidate dicts.

  persist_candidates(db, product, source_document, candidates) -> int
    Write one ProductAttribute row per candidate with
    status='proposed', source_id, extraction_method from source doc_type,
    model_used=settings.GEMINI_MODEL. Respect the
    UNIQUE(product_id, key, source_id) constraint — update on conflict.
```

```
  enrich_product(db, product_id) -> dict
    Orchestrate: load schema -> for each SourceDocument -> extract -> persist.
    Set product.status through 'enriching' then 'needs_review'.
    On failure set status='failed' and write the reason to product.error.
    Always record enriched_at and model_used. Never raise to the caller.

ERROR HANDLING — mandatory
  Wrap every Gemini call. On any failure record the reason on the product and
  continue to the next source. One bad PDF must never fail a whole job.

TASK — add the background job, copying the existing pattern exactly.
Read routes/campaigns.py research-leads-async and its _run_research_job to see
the shape: create job row, return job_id immediately, BackgroundTasks worker,
increment processed/succeeded/failed as it goes, poll endpoint.

  POST /catalogs/{id}/enrich-async?limit=50   -> {job_id, poll_url, ...}
  GET  /catalogs/enrichment-job/{job_id}      -> progress

DO NOT TOUCH  services/knowledge_service.py, services/embedding_service.py

ACCEPTANCE
  - A product with one spec-sheet PDF yields 15+ ProductAttribute rows
  - Every row has a non-null source_id and a confidence between 0 and 100
  - Deleting the Gemini API key makes enrichment fail gracefully with the
    reason on product.error, and the app keeps serving
  - Enriching 25 products returns a job_id in under 500 ms
```

PROMPT 4 · Normalisation Agent

```
CONTEXT
SpecForge. ProductAttribute rows carry value_raw (verbatim from source) and
empty value_norm and unit.

TASK — create services/normalization_service.py.
THIS MODULE MUST CONTAIN NO AI CALLS. Pure deterministic Python. This is a
correctness guarantee we make on the deck, so do not import any model client.

Implement, each returning (value_norm: str|None, unit: str|None, note: str|None):

  normalize_length(raw)
    Handle: 1/2", 1/2 in, 0.5 inch, .5", 12.7mm, 12,7 mm, 1-1/2", 1 1/2 in,
            3/4 NPT, 25 cm, 2 ft
    Canonical output: millimetres as a decimal string, unit "mm".
    Fractions and mixed numbers must be parsed exactly (1-1/2 -> 38.1).

  normalize_pressure(raw)
    Handle: 600 WOG, 600WOG, 150 PSI, 150#, 10 bar, 1000 kPa, 2.5 MPa
    Canonical: psi. Note that WOG and CWP are psi ratings, not separate units.

  normalize_temperature(raw)
    Handle: -20F, -20 °F, 400 degF, 200C, -28.9 °C, ranges like "-20F to 400F"
    Canonical: Celsius. For a range return the two ends separately —
    the caller splits into _min and _max keys.

  normalize_mass(raw)         lb, lbs, kg, g, oz  -> kg
  normalize_thread(raw)       NPT, FNPT, MNPT, BSP, BSPT, NPSM, UNF, UNC
                              -> canonical lowercase token, e.g. "npt_female"
  normalize_material(raw)     SS304, 304 SS, SS 304, Stainless Steel 304,
                              T304, bronze, brass, cast iron, PVC, CPVC, PTFE
                              -> canonical lowercase token e.g. "stainless_304"
  normalize_boolean(raw)      Yes/Y/True/1/✓/X -> "true"; No/N/False/0/- -> "false"
```

```
  normalize_enum(raw, allowed_values)
                              case-insensitive, whitespace- and
                              punctuation-insensitive match against
                              allowed_values; return the canonical form or None

  normalize_value(raw, unit_family, data_type, allowed_values) -> dict
    Dispatch on unit_family, falling back to enum/boolean/string by data_type.
    Return {value_norm, unit, note}.

  normalize_product_attributes(db, product_id) -> dict
    Load the schema, iterate the product's ProductAttribute rows, populate
    value_norm and unit. When normalisation fails, leave value_norm NULL and
    append "normalization_failed:<reason>" to validation_flags.
    Return {normalized, failed, skipped}.

REQUIREMENTS
  - Never mutate value_raw. It is the audit record.
  - Round to 4 decimal places max; never use float repr directly in output.
  - Every conversion factor must be a named module-level constant with a
    comment citing the conversion, e.g. INCH_TO_MM = 25.4
  - Write tests in backend/tests/test_normalization.py covering at minimum:
      1/2" == 12.7mm ; 1-1/2" == 38.1mm ; 600 WOG == 600 psi ;
      -20F == -28.8889C ; SS304 == stainless_304 ; 10 bar == 145.0377 psi
    Use pytest. This is the one place in the project where tests are mandatory,
    because unit maths is the thing we claim never to get wrong.

Hook it into enrich_product() so normalisation runs immediately after
persistence, before scoring.

ACCEPTANCE
  - pytest backend/tests/test_normalization.py passes
  - grep for "genai\|gemini\|openai" in normalization_service.py returns nothing
  - A product with "1/2 in" and another with "12.7mm" show identical value_norm
```

PROMPT 5 · Validation Agent and conflict detection

```
CONTEXT
SpecForge. Attributes now carry value_raw, value_norm and unit.
AttributeConflict model exists and is unused.

TASK A — services/validation_service.py, rule-based. NO AI IN THIS FUNCTION SET.

  validate_attribute(attribute, schema_entry) -> list[str]
    Return flag strings, empty when clean:
      "missing_required"          required and value_norm is null
      "type_mismatch"             number expected, value_norm not numeric
      "enum_violation"            not in allowed_values
      "out_of_range:min|max"      outside numeric bounds
      "unit_family_mismatch"      unit does not belong to the declared family
      "normalization_failed"      preserved from phase 4
      "low_confidence"            confidence < 60
    Each flag that fires reduces confidence by 15, floored at 5.

  validate_product(db, product_id) -> dict
    Run every attribute, persist validation_flags, and detect missing required
    attributes by creating a placeholder ProductAttribute with
    value_raw=null, status='proposed', flags=["missing_required"].
    Return {valid, flagged, missing_required}.

TASK B — AI plausibility pass, a separate function so the boundary is explicit.

  ai_plausibility_check(db, product_id) -> dict
```

```
    Build a compact table of the product's normalised attributes and ask Gemini
    a single question: given this product category and these values, flag any
    that are physically or commercially implausible, with a one-line reason.
    Demand strict JSON: [{"key":"...","implausible":true,"reason":"..."}]
    Append "ai_implausible:<reason>" to validation_flags for each hit and drop
    confidence by 20.
    On any model failure, log and return {"skipped": true}. Never raise.
    This must run AFTER rule validation so rules are never blocked by the model.

TASK C — conflict detection.

  detect_conflicts(db, product_id) -> list[AttributeConflict]
    Group the product's ProductAttribute rows by key. A key with candidates
    from 2+ distinct source_ids is a conflict when the normalised values
    disagree — compare value_norm, and for numerics allow 1% tolerance before
    calling it a disagreement.
    Create one AttributeConflict per disagreeing key with every candidate and
    its source and confidence.
    Auto-resolve when one candidate's confidence exceeds every other by 25 or
    more AND its source doc_type is 'pdf' (a spec sheet outranks a web page):
    set resolution='auto', resolved_value, and mark the losing rows
    status='rejected'. Otherwise leave resolution='unresolved' and set every
    candidate row to status='conflicted'.

    Port the source-quality heuristics from the old
    services/lead_discovery_service.py (_role_quality, _url_quality style
    scoring) — the same ranked-preference logic applies to document sources.

  Endpoints:
    GET   /products/{id}/conflicts
    PATCH /products/{id}/conflicts/{conflict_id}   {resolved_value}
          -> sets resolution='human', approves the matching attribute row,
             rejects the others

Wire validate_product -> ai_plausibility_check -> detect_conflicts into
enrich_product(), after normalisation.

ACCEPTANCE
  - Manually setting a pressure value to 99999 raises "out_of_range:max"
  - Two sources giving different body_material create one AttributeConflict
    with 2 candidates and resolution='unresolved'
  - A spec-sheet value at confidence 95 against a web value at 60 auto-resolves
  - Removing the Gemini key still runs rule validation and conflict detection
```

PROMPT 6 · Vision extraction

```
CONTEXT
SpecForge. Extraction currently reads text only. google-genai is installed and
GEMINI_MODEL is a multimodal Gemini 2.5 Flash model — no new dependency is
needed for vision. pypdf is installed. Add pymupdf (fitz) for page rendering.

TASK — extend extraction to images and scanned PDFs.

1. services/vision_service.py

   pdf_has_text_layer(file_path) -> bool
     Extract text with pypdf. Fewer than 100 characters per page on average
     means it is a scan.

   render_pdf_pages(file_path, max_pages=10) -> list[(page_number, png_bytes)]
     Render with pymupdf at 200 DPI.
```

```
   extract_from_image(product, schema, image_bytes, source_label) -> list[dict]
     Send the image to Gemini with a prompt that:
       - states the product identity and lists the schema attributes
       - instructs it to read specification tables, callouts and dimension
         drawings
       - for a multi-row catalog table, extract ONLY the row matching this
         product's part number, and say so explicitly
       - returns the same strict JSON contract as the text extractor, with an
         extra field "visual_evidence" describing where on the page the value
         was found
       - never guesses from product appearance

2. Wire into extraction_service.extract_from_source:
     doc_type == 'image'                    -> vision path
     doc_type == 'pdf' and no text layer    -> render pages, vision each
     doc_type == 'pdf' with a text layer    -> existing text path
     Optionally, for a text-layer PDF where text extraction produced fewer
     than 5 attributes, fall back to vision on the first 3 pages.

   Set extraction_method='vision' and page_number on every attribute produced
   this way, so provenance shows both the file and the page.

3. Cost and latency controls
     Cap vision at 10 pages per document, configurable via
     VISION_MAX_PAGES in core/config.py.
     Cache by SourceDocument.content_hash + page_number so re-running a job
     never re-bills the same page.

ACCEPTANCE
  - A scanned spec sheet with no text layer produces attributes
  - Those attributes show extraction_method='vision' and a page_number
  - A digital PDF still uses the text path (verify by checking
    extraction_method='pdf')
  - Re-running enrichment on the same document makes zero new vision calls
```

PROMPT 7 · Quality scoring

```
CONTEXT
SpecForge. Attributes are extracted, normalised, validated and deconflicted.
The old services/lead_scoring_service.py contains a scoring pattern worth
studying — clamp helpers, banded grading, reason strings.

TASK — create services/quality_service.py.

  compute_attribute_confidence(attribute, schema_entry, source) -> int
    Start from the model's extracted confidence, then adjust:
      +10  source doc_type == 'pdf'            (spec sheet is authoritative)
      +5   extraction_method == 'html'
      -10  extraction_method == 'inferred'
      +10  corroborated by 2+ sources agreeing
      -15  per validation flag
      -20  status == 'conflicted'
    Clamp 0-100.

  compute_product_scores(db, product_id) -> dict
    completeness_score = round(100 * required_attrs_filled / required_attrs_total)
      where "filled" means value_norm is not null and status != 'rejected'
    confidence_score   = mean confidence of non-rejected attributes
    quality_grade      A >=90 completeness and >=85 confidence
                       B >=75 and >=70
                       C >=50 and >=50
                       D otherwise
```

```
    needs_review_count = attributes with status in (proposed, conflicted)
                         AND (confidence < 75 OR validation_flags non-empty)
    Persist to the product. Set status='approved' when every attribute is
    approved, otherwise 'needs_review'.

  score_catalog(db, catalog_id) -> dict
    Aggregate: product count by grade, mean completeness, mean confidence,
    total attributes, total conflicts, review backlog size.

  Endpoints
    POST /quality/product/{id}/score
    GET  /quality/catalog/{id}/summary
    GET  /dashboard/stats            -- rewrite the existing one to return
         catalogs, products, products_by_status, products_by_grade,
         mean_completeness, mean_confidence, attributes_total,
         attributes_approved, conflicts_open, review_backlog,
         recent_products, enrichment_funnel
         (funnel = ingested -> sourced -> extracted -> validated -> approved)

Call compute_product_scores at the end of enrich_product().

ACCEPTANCE
  - Every enriched product shows completeness, confidence and a grade
  - A product with all required attributes at high confidence grades A
  - GET /dashboard/stats returns the funnel with five non-zero stages
```

PROMPT 8 · Review Queue UI

```
CONTEXT
SpecForge frontend. React 19 + Vite + Tailwind 4 + React Router 7 + axios +
Recharts + Framer Motion. There is an existing design system you must reuse
without modification:
  components/ui/  Button, Card, Badge, Table, PageHeader, EmptyState,
                  Skeleton, Toast, ThemeToggle
  components/     StatCard, Sidebar, Navbar
  src/index.css   CSS-variable design tokens; classes: glass, glass-hover,
                  well, field, label, btn/btn-primary/btn-secondary/btn-ghost/
                  btn-tone, tone + tone-success|info|violet|pink|warn|danger|
                  neutral, text-ink, text-ink-2, text-muted, text-faint,
                  surface-1/2/3, surface-sunk, line-1/2/3, elev-1/2/3,
                  bg-accent, bg-<tone>-soft, bg-<tone>-solid, border-<tone>-soft
NEVER hard-code a colour or use a raw Tailwind palette class such as
text-slate-500 or bg-emerald-50. Only the tokens above. Both light and dark
themes must work.

TASK — rebuild the pages for the product domain.

RENAME  pages/Campaigns.jsx -> Catalogs.jsx
        pages/Leads.jsx     -> Products.jsx
        pages/LeadDiscovery.jsx -> Sources.jsx
Update App.jsx routes and Sidebar.jsx nav to:
  Dashboard / Catalogs / Products / Sources / Review / Schema / Knowledge / Settings

1. pages/Products.jsx
   Catalog selector. Table columns: part number, manufacturer, category,
   completeness bar, confidence, grade badge, status badge, needs-review count.
   Filters: status, grade, min confidence, needs-review only, text search.
   Bulk actions: enrich selected, export selected.
   "Enrich Catalog" card that POSTs the async job and polls the progress
   endpoint every 3 s, rendering a progress bar — copy the polling pattern
   from the old LeadAgentLauncher.jsx / EmailExtraction.jsx.
```

```
2. pages/ProductDetail.jsx   (new route /products/:id)
   Header: part number, manufacturer, grade, completeness, confidence.
   Attribute table, one row per attribute:
     key · value_raw · → · value_norm + unit · confidence chip ·
     extraction-method badge · source link · flag chips · status badge
   Clicking the source opens a right-side drawer showing the document name,
   page number and text_snippet, with a link to the stored file. THIS IS THE
   MOST IMPORTANT INTERACTION IN THE PRODUCT — make it fast and obvious.
   A conflicts section listing every unresolved conflict with its candidates
   side by side and a one-click resolve.

3. pages/Review.jsx   (new)
   The human-in-the-loop queue. Cross-catalog list of every attribute with
   status proposed or conflicted AND (confidence < 75 or flags non-empty).
   Sort by confidence ascending — worst first.
   Per row: approve, reject, or edit value_norm inline then approve.
   Bulk: select all above a confidence threshold and approve in one action.
   Keyboard: A approve, R reject, J/K move. Show a hint bar.
   Optimistic UI with rollback on error.

4. pages/Schema.jsx   (new)
   List AttributeSchemas per catalog. Editor for a category's attribute list
   — key, label, data type, unit family, allowed values, required, min, max.
   JSON import/export.

5. pages/Dashboard.jsx
   Rewrite the stats for the new /dashboard/stats payload. Four StatCards:
   products, mean completeness, mean confidence, review backlog.
   Recharts: the 5-stage enrichment funnel as a bar chart, grade distribution
   as a second chart. Charts must read var(--chart-1) .. var(--chart-5).

DELETE pages/Opportunities.jsx and pages/Calls.jsx if still present.

DO NOT TOUCH  components/ui/*, index.css, tailwind.config.js,
              theme/*, services/api.js

ACCEPTANCE
  - npm run build succeeds with no warnings about unknown classes
  - Every page renders correctly in both light and dark
  - Review queue approve/reject updates optimistically and persists
  - Clicking an attribute's source opens the drawer with page number
  - Mobile at 390px wide is usable on every page
```

PROMPT 9 · Export, schema generator, polish

```
CONTEXT
SpecForge, feature-complete through review. Two additions and a cleanup.

TASK A — export. services/export_service.py

  export_catalog_csv(db, catalog_id, approved_only=True) -> bytes
    Wide format: one row per product, one column per schema attribute key,
    values from value_norm. Include part_number, manufacturer, category,
    completeness_score, confidence_score, quality_grade.
    Add a companion column "<key>__source" holding "filename p.N" so the
    export itself carries provenance.

  export_catalog_json(db, catalog_id, approved_only=True) -> bytes
    Nested: product object with an attributes array, each entry carrying
    value_raw, value_norm, unit, confidence, status, extraction_method and a
    source object {filename, page_number, url}.
```

```
  Endpoints
    GET /catalogs/{id}/export.csv?approved_only=true
    GET /catalogs/{id}/export.json?approved_only=true
    Stream with correct Content-Disposition so the browser downloads.

TASK B — AI schema generator.
The deleted services/opportunity_service.py used Gemini to turn a short
free-text goal into ~20 structured fields. Recreate that pattern as
services/schema_service.py:

  generate_attribute_schema(db, catalog_id, category_name, sample_text=None) -> dict
    Given a category like "Ball Valve" — optionally plus text from one sample
    spec sheet — ask Gemini to propose the attribute list a distributor would
    need: key, label, data_type, unit_family, allowed_values, required, min, max.
    Return strict JSON matching the AttributeSchema.attributes shape.
    Save as a draft the user edits on the Schema page.

  POST /schemas/generate  {catalog_id, category_name, source_document_id?}

TASK C — cleanup and hardening
  - Add a simple API key guard: if API_KEY is set in the environment, require
    header X-API-Key on every mutating route (POST/PATCH/DELETE). Read-only
    routes stay open so judges can browse. Document this in the README.
  - Add a /health/deep endpoint checking DB connectivity, pgvector presence
    and Gemini reachability, returning per-check status.
  - Update README.md: what SpecForge is, architecture diagram, setup, env vars,
    and an explicit statement that it reuses an internal framework the team
    wrote previously.
  - Remove every remaining reference to leads, campaigns, emails or outreach
    in code, comments and docs. grep to prove it.

ACCEPTANCE
  - Export CSV opens in Excel with provenance columns populated
  - Schema generation returns a valid editable schema for "Ball Valve"
  - With API_KEY set, POST without the header returns 401; GET still works
  - grep -ri "lead\|campaign\|outreach\|email draft" backend/app frontend/src
    returns nothing outside the git history
```

PROMPT 10 · Deploy and seed

```
CONTEXT
SpecForge, feature-complete. Deploying to Vercel (frontend), Render (backend)
and Supabase (Postgres with pgvector).

TASK

1. backend/scripts/seed_demo.py
   Idempotent seeding script that:
     - creates catalog "Industrial Valves & Fittings"
     - creates AttributeSchemas for ball_valve, pressure_gauge, pipe_fitting
       (read them from backend/scripts/schemas/*.json so they are editable)
     - ingests backend/scripts/demo_data/products.csv
     - registers every PDF in backend/scripts/demo_data/docs/ against its
       product by matching part number in the filename
     - runs the full enrichment pipeline synchronously
     - approves every attribute with confidence >= 85, leaving the rest for the
       demo review queue
     - leaves exactly 2 unresolved conflicts and 5 review-queue items so the
       demo has something to show
   Run with: python -m scripts.seed_demo --reset

2. Deployment configuration
```

```
   - render.yaml with build and start commands, health check path /health
   - Ensure CREATE EXTENSION IF NOT EXISTS vector runs on Supabase before
     first boot; document it in README
   - frontend/.env.production with VITE_API_BASE_URL pointing at Render
   - Verify CORS allows the Vercel domain via FRONTEND_URLS

3. Cold-start mitigation
   Add backend/scripts/keepalive.md documenting that Render free tier sleeps
   after ~15 minutes and that an external uptime pinger should hit /health
   every 10 minutes during judging.

4. Final verification checklist as backend/scripts/preflight.sh
   Curl each critical endpoint against production and assert a 200:
     /health  /health/deep  /dashboard/stats  /catalogs  /products
     /catalogs/1/export.csv
   Print a pass/fail table.

ACCEPTANCE
  - Fresh Supabase database + seed script -> a fully populated demo in one command
  - Production URL loads, all six pages render, theme toggle works
  - preflight.sh passes every check against production
  - Enrichment of 5 new products works end to end on production
```

PART 8 · THE 3-MINUTE DEMO VIDEO

The template caps it at 3 minutes. That is tight — every second must earn its place. Do **not** narrate the UI; narrate the problem being solved.

Time	On screen	Say
0:00–0:20	A spreadsheet row: part number, brand, one line. Then a distributor's product page with 17 blank fields.	"This is what a manufacturer sends. This is what it has to become. Between them sits 15 minutes of skilled human work, times five thousand SKUs."
0:20–0:40	Upload the CSV. Upload a folder of spec-sheet PDFs. Click **Enrich Catalog**.	"SpecForge takes the three fields you actually have, plus whatever documents came with them."
0:40–1:00	Job progress bar climbing. Products filling in live.	"Five agents run per product — sourcing, extraction, normalisation, validation, review. It's a background job, so twenty-five SKUs and twenty-five thousand are the same architecture."
1:00–1:40	Open a product. 20+ attributes. **Click one**	"Every value carries a receipt. This pressure rating came

Time	On screen	Say
	attribute's source — the drawer opens on the exact PDF page.	from page two of this spec sheet — here's the sentence it came from. Not a confidence number you have to take on faith. The actual source."
1:40–2:05	Show raw `1/2 in` next to normalised `12.7 mm`. Then a conflict card with two disagreeing sources.	"Units are converted in Python, never by the model — we don't ask a language model to do arithmetic. And when two sources disagree, we don't pick one silently. We raise it."
2:05–2:30	Review queue. Bulk-approve everything above 85. Hand-edit one low-confidence value.	"Nothing publishes itself. The human reviews the uncertain few percent and approves the rest in one click."
2:30–2:50	Export CSV. Open it. Show the provenance columns.	"Out comes a clean catalog — and every column has a matching source column, so the audit trail survives the export."
2:50–3:00	Dashboard funnel.	"From three fields to a sellable catalog, with a receipt for every value."

The moment that wins it is at 1:00. Clicking an attribute and landing on the exact PDF page is something no other team will show. Rehearse that transition until it is instant, and give it the full 40 seconds.

PART 9 · RISKS AND FINAL CHECKS

Technical risks

Risk	Mitigation
Extraction quality is the real work — not the plumbing	Budget a full day purely on prompt iteration in Phase 3. Build an eval sheet: 10 products × known-correct attributes, and measure

Risk	Mitigation
	accuracy after each prompt change. This is the difference between a demo that works and one that works *reliably*.
Render cold start — free tier sleeps after ~15 min	Set up an external uptime pinger on `/health` every 10 minutes for the whole judging window. Judges click the prototype link at random times.
Scraping distributor sites fails — Grainger, McMaster block bots and your scraper honours robots.txt	Already designed around: documents are the primary input, URLs are secondary. Do not fight this.
Vision cost and latency	Cap at 10 pages per document, cache by `content_hash` + page number. Never re-bill a page.
Demo data quality	Real part numbers with real spec sheets. Curate before you build, not after. Fake data produces a demo that a Unilog judge will see straight through.
pgvector missing on Supabase	`CREATE EXTENSION IF NOT EXISTS vector` before first boot. Keyword search is the fallback, but semantic taxonomy matching is part of the story.

Process risks

Eligibility — settle it in the first hour. Find out whether UniHack requires code written during the event. Fresh repo regardless, plus an explicit line in the README and on the deck. If the rules forbid reuse outright, the phase plan still works — it just becomes a genuine from-scratch build, and you would drop Phases 6 and 9B.

Domain naivety. None of you work in industrial distribution. Before Phase 1, spend an hour reading real product pages and spec sheets. Learn what a distributor actually calls things. Building something that looks right to you and wrong to a Unilog judge is the single likeliest failure mode.

Scope discipline. If something has to give, drop in this order: schema generator (9B) → cross-reference matching → vision (6). **Never drop normalisation or provenance.** They are the two things that make the trust story provable, and trust is where the marks are.

Pre-submission checklist

Repository - [ ] Public, and verified in a private browsing window - [ ] README explains what it is, how to run it, and discloses the reused framework - [ ] No `.env`, no credentials, no database

file committed — `git log --all -- '*.env'` returns nothing - [ ] Architecture diagram in the README - [ ] Commits from all three team members, on `main`, with GitHub-linked emails

Prototype - [ ] Live URL loads in a private window with no login - [ ] Backend warm, uptime pinger running - [ ] Demo catalog seeded — 25 products, enriched, 2 conflicts, 5 review items - [ ] Every page renders in light and dark - [ ] Works on a phone

Deck - [ ] All 15 template slides filled, none left as placeholder text - [ ] Slide 4 answers all three questions, and names all five validation layers in Q2 - [ ] Architecture and process-flow diagrams are readable at presentation size - [ ] Slide 14 links all tested from a private window

Video - [ ] Under 3 minutes - [ ] Audio is clear — bad audio loses more marks than bad visuals - [ ] The source-provenance click is in it, unmissable - [ ] Uploaded unlisted, link tested while signed out

---

The one-sentence pitch

> **SpecForge turns the three fields a manufacturer actually gives you into a complete, validated catalog record — and every single value comes with a receipt pointing at the exact page it came from.**